Execution Time, Memory & Complexity

Lab 3 — Performance Measurement and Complexity Comparison

Name	Kushagra Verma
Roll No.	2401201044
Specialisation	BCA (AI & Data Science)
Section	_____

Lab Name: Lab 3 — Smart Resource Allocation and Scheduling System using Greedy Algorithms

Measurement method

Execution time was measured with time.perf_counter(); peak traced Python allocation was measured with tracemalloc. Each case was run five times and the median was reported. Values are machine- and runtime-dependent; they are measured results from this execution, not universal constants.

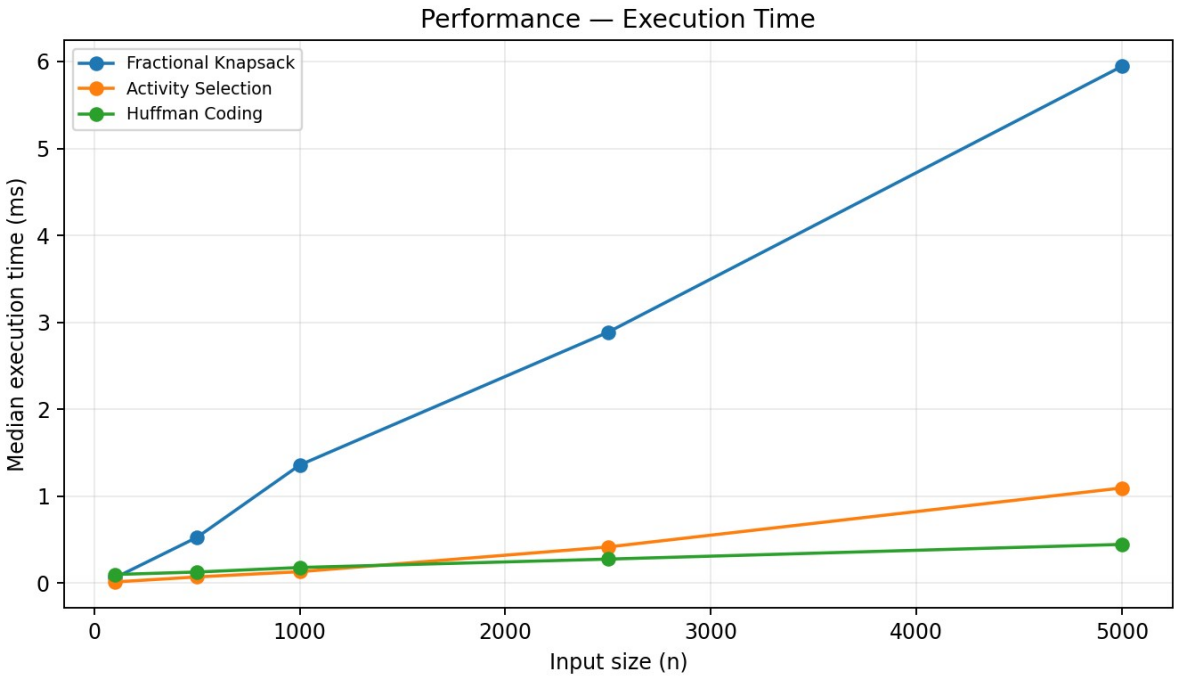
Measured results

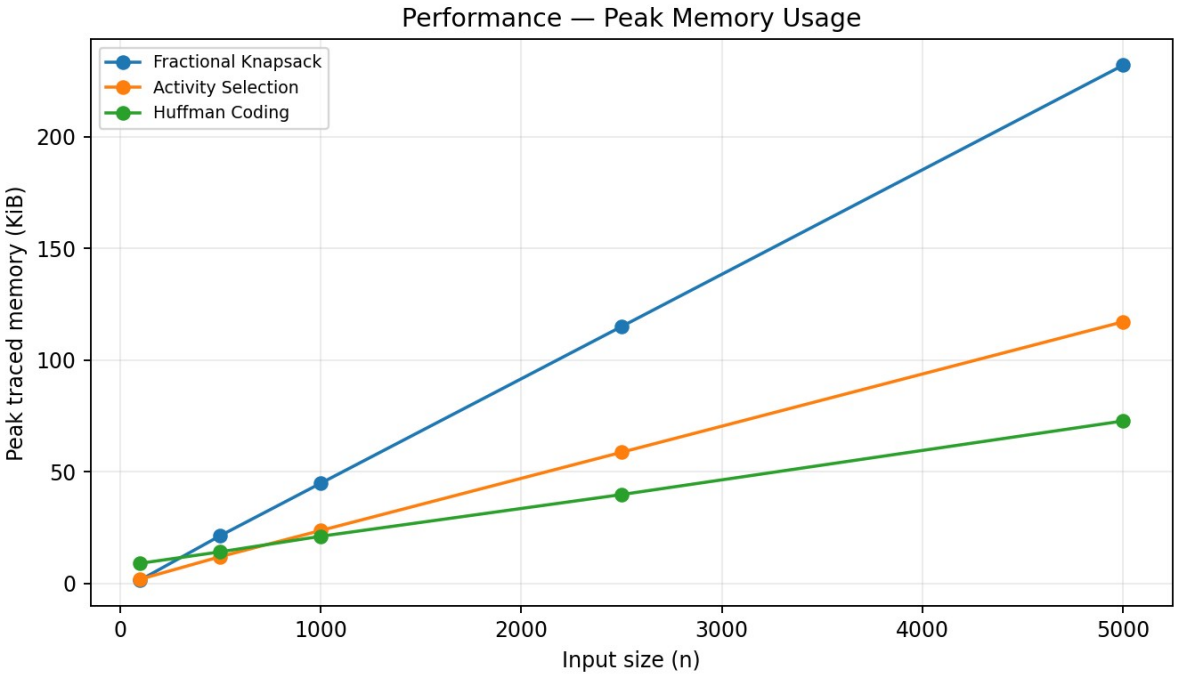
Algorithm	Input n	Median time (ms)	Peak memory (KiB)
Fractional Knapsack	100	0.0596	1.41
Activity Selection	100	0.0128	1.79
Huffman Coding	100	0.0983	8.92
Fractional Knapsack	500	0.5260	21.32
Activity Selection	500	0.0703	11.91
Huffman Coding	500	0.1269	14.07
Fractional Knapsack	1000	1.3580	44.77
Activity Selection	1000	0.1309	23.60
Huffman Coding	1000	0.1788	21.03
Fractional Knapsack	2500	2.8864	114.98
Activity Selection	2500	0.4153	58.66
Huffman Coding	2500	0.2755	39.72
Fractional Knapsack	5000	5.9446	232.02
Activity Selection	5000	1.0939	117.10
Huffman Coding	5000	0.4448	72.68

Asymptotic complexity

Algorithm	Time complexity	Auxiliary space	Reason
Fractional Knapsack	O(n log n)	O(n)	Sorting by value/weight ratio dominates.
Activity Selection	O(n log n)	O(n)	Sorting by finish time dominates; scan is O(n).
Huffman Coding	O(k log k)	O(k)	k distinct symbols; heap operations and tree construction dominate.

Performance graph





Observation

Fractional Knapsack and Activity Selection show the expected sorting-related growth. Huffman timings depend strongly on the number of distinct symbols (k), not only the raw text length. The memory values are Python-traced peak allocations and do not include every native/runtime allocation.